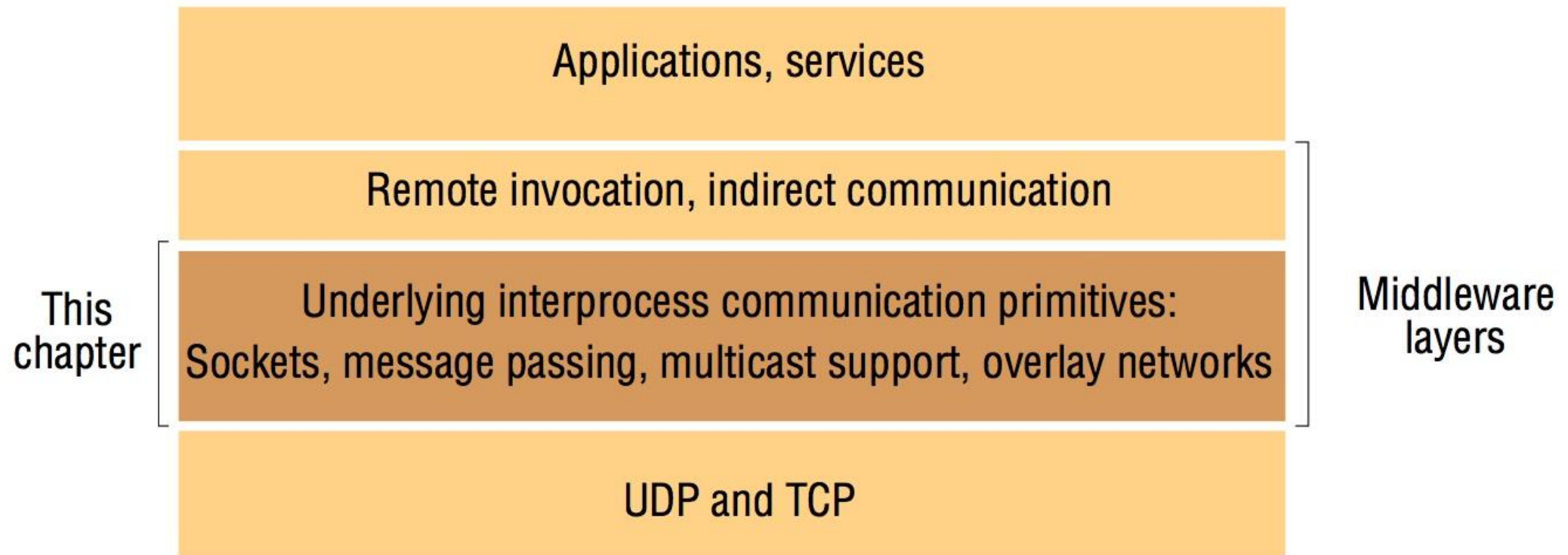

PARALLEL AND DISTRIBUTED COMPUTING

CHAPTER 4: INTERPROCESS COMMUNICATION

LESSON TOPICS

- Introduction
- The API for the Internet protocols
- External data representation and marshalling
- Multicast communication
- Network Virtualization: Overlay networks

MIDDLEWARE LAYERS



CHARACTERISTICS OF INTERPROCESS COMMUNICATION

- Message passing between a pair of processes can be supported by two message communication operations, send and receive, defined in terms of destinations and messages.
- To communicate, one process sends a message (a sequence of bytes) to a destination and another process at the destination receives the message.
- Synchronous and asynchronous communication
 - A queue is associated with each message destination. Sending processes cause messages to be added to remote queues and receiving processes remove messages from local queues.

API FOR THE INTERNET PROTOCOLS

CHARACTERISTICS OF INTERPROCESS COMMUNICATION

- In the **synchronous** form of communication, the sending and receiving processes synchronize at every message. In this case, both send and receive are blocking operations. Whenever a send is issued the sending process (or thread) is blocked until the corresponding receive is issued. Whenever a receive is issued by a process (or thread), it blocks until a message arrives.
- In the **asynchronous** form of communication, the use of the send operation is non blocking in that the sending process is allowed to proceed as soon as the message has been copied to a local buffer, and the transmission of the message proceeds in parallel with the sending process. The receive operation can have blocking and non-blocking variants.

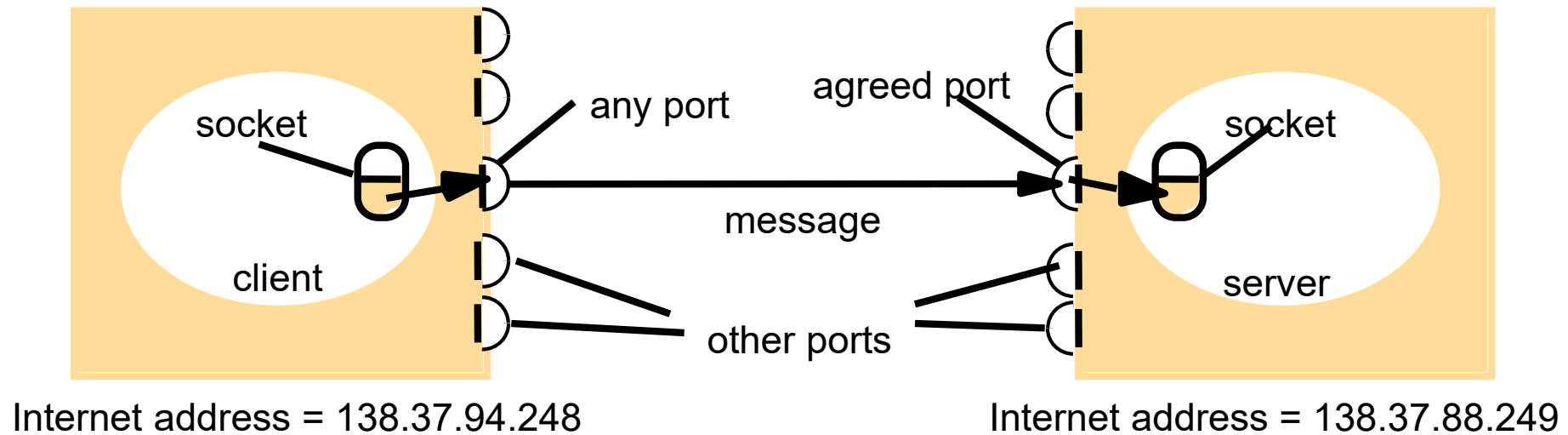
CHARACTERISTICS OF INTERPROCESS COMMUNICATION

- **Message destinations** : in the Internet protocols, messages are sent to (Internet address, local port) pairs. A local port is a message destination within a computer, specified as an integer. A port has exactly one receiver (multicast ports are an exception) but can have many senders. Processes may use multiple ports to receive messages. Any process that knows the number of a port can send a message to it.
- **Reliability**: defines reliable communication in terms of validity and integrity. As far as the validity property is concerned, a point-to-point message service can be described as reliable if messages are guaranteed to be delivered despite a ‘reasonable’ number of packets being dropped or lost. For integrity, messages must arrive uncorrupted and without duplication.
- **Ordering** • Some applications require that messages be delivered in sender order – that is, the order in which they were transmitted by the sender. The delivery of messages out of sender order is regarded as a failure by such applications.

SOCKETS AND PORTS

- Both forms of communication (UDP and TCP) use the socket abstraction, which provides an endpoint for communication between processes.
- Sockets originate from BSD UNIX but are also present in most other versions of UNIX, including Linux as well as Windows and the Macintosh OS.
- Interprocess communication consists of transmitting a message between a socket in one process and a socket in another process.
- For a process to receive messages, its socket must be bound to a local port and one of the Internet addresses of the computer on which it runs.
- Messages sent to a particular Internet address and port number can be received only by a process whose socket is associated with that Internet address and port number.
- Processes may use the same socket for sending and receiving messages. Each computer has a large number (2^{16}) of possible port numbers for use by local processes for receiving messages.

SOCKETS AND PORTS



UDP DATAGRAM COMMUNICATION

- A datagram sent by UDP is transmitted from a sending process to a receiving process without acknowledgement or retries. If a failure occurs, the message may not arrive.
- The following are some issues relating to datagram communication:
 - Message size: The receiving process needs to specify an array of bytes of a particular size in which to receive a message. If the message is too big for the array, it is truncated on arrival.
 - Blocking: Sockets normally provide non-blocking sends and blocking receives for datagram communication (a non-blocking receive is an option in some implementations).
 - Timeouts: The receive that blocks forever is suitable for use by a server that is waiting to receive requests from its clients. But in some programs, it is not appropriate that a process that has invoked a receive operation should wait indefinitely in situations where the sending process may have crashed or the expected message may have been lost. To allow for such requirements, timeouts can be set on sockets.

FAILURE MODEL FOR UDP DATAGRAMS

- **Omission failures:** Messages may be dropped occasionally, either because of a checksum error or because no buffer space is available at the source or destination.
- **Ordering:** Messages can sometimes be delivered out of sender order.
- Use of UDP
 - DNS
 - VOIP and other video/audio
 - Higher-level multicast with stronger properties

UDP CLIENT SENDS A MESSAGE TO THE SERVER AND GETS A REPLY

```
import java.net.*;
import java.io.*;
public class UDPClient{
    public static void main(String args[]){
        // args give message contents and server hostname
        DatagramSocket aSocket = null;
        try {
            aSocket = new DatagramSocket();
            byte [] m = args[0].getBytes();
            InetAddress aHost = InetAddress.getByName(args[1]);
            int serverPort = 6789;
            DatagramPacket request = new DatagramPacket(m, m.length(), aHost, serverPort);
            aSocket.send(request);
            byte[] buffer = new byte[1000];
            DatagramPacket reply = new DatagramPacket(buffer, buffer.length);
            aSocket.receive(reply);
            System.out.println("Reply: " + new String(reply.getData()));
        }catch (SocketException e){System.out.println("Socket: " + e.getMessage());}
        }catch (IOException e){System.out.println("IO: " + e.getMessage());}
    }finally {if(aSocket != null) aSocket.close();}
}
```

UDP SERVER REPEATEDLY RECEIVES A REQUEST AND SENDS IT BACK TO THE CLIENT

```
import java.net.*;
import java.io.*;
public class UDPServer{
    public static void main(String args[]){
        DatagramSocket aSocket = null;
        try{
            aSocket = new DatagramSocket(6789);
            byte[] buffer = new byte[1000];
            while(true){
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                aSocket.receive(request);
                DatagramPacket reply = new DatagramPacket(request.getData(),
                    request.getLength(), request.getAddress(), request.getPort());
                aSocket.send(reply);
            }
        }catch (SocketException e){System.out.println("Socket: " + e.getMessage());}
        }catch (IOException e) {System.out.println("IO: " + e.getMessage());}
    }finally {if(aSocket != null) aSocket.close();}
}
```

TCP STREAM COMMUNICATION

- Message sizes: The application can choose how much data it writes to a stream or reads from it. It may deal in very small or very large sets of data. The underlying implementation of a TCP stream decides how much data to collect before transmitting it as one or more IP packets. On arrival, the data is handed to the application as requested.
- Lost messages: The TCP protocol uses an acknowledgement scheme.
- Flow control: The TCP protocol attempts to match the speeds of the processes that read from and write to a stream.

TCP STREAM COMMUNICATION

- Message duplication and ordering: Message identifiers are associated with each IP packet, which enables the recipient to detect and reject duplicates, or to reorder messages that do not arrive in sender order.
- Message destinations: A pair of communicating processes establish a connection before they can communicate over a stream. Once a connection is established, the processes simply read from and write to the stream without needing to use Internet addresses and ports. Establishing a connection involves a connect request from client to server followed by an accept request from server to client before any communication can take place. This could be a considerable overhead for a single client-server request and reply.

TCP STREAM COMMUNICATION

- The API for stream communication assumes that when a pair of processes are establishing a connection, one of them plays the client role and the other plays the server role, but thereafter they could be peers.
- The client role involves creating a stream socket bound to any port and then making a connect request asking for a connection to a server at its server port.
- The server role involves creating a listening socket bound to a server port and waiting for clients to request connections. The listening socket maintains a queue of incoming connection requests.
- In the socket model, when the server accepts a connection, a new stream socket is created for the server to communicate with a client, meanwhile retaining its socket at the server port for listening for connect requests from other clients.
- The pair of sockets in the client and server are connected by a pair of streams, one in each direction.
- Thus each socket has an input stream and an output stream.
- One of the pair of processes can send information to the other by writing to its output stream, and the other process obtains the information by reading from its input stream.
- When an application closes a socket, this indicates that it will not write any more data to its output stream.

TCP STREAM COMMUNICATION

- some outstanding issues related to stream communication:
 - Matching of data items: Two communicating processes need to agree as to the contents of the data transmitted over a stream.
 - Blocking: The data written to a stream is kept in a queue at the destination socket. When a process attempts to read data from an input channel, it will get data from the queue or it will block until data becomes available.
 - Threads: When a server accepts a connection, it generally creates a new thread in which to communicate with the new client. The advantage of using a separate thread for each client is that the server can block when waiting for input without delaying other clients.

TCP STREAM COMMUNICATION

Failure model

- To satisfy the integrity property of reliable communication, TCP streams use checksums to detect and reject corrupt packets and sequence numbers to detect and reject duplicate packets.
- For the sake of the validity property, TCP streams use timeouts and retransmissions to deal with lost packets. Therefore, messages are guaranteed to be delivered even when some of the underlying packets are lost.

TCP STREAM COMMUNICATION

- When a connection is broken, a process using it will be notified if it attempts to read or write.
- This has the following effects:
 - The processes using the connection cannot distinguish between network failure and failure of the process at the other end of the connection.
 - The communicating processes cannot tell whether the messages they have sent recently have been received or not.

Uses of TCP: HTTP, FTP, Telnet, SMTP

TCP CLIENT MAKES CONNECTION TO SERVER, SENDS REQUEST AND RECEIVES REPLY

```
import java.net.*;
import java.io.*;
public class TCPClient {
    public static void main (String args[]) {
        // arguments supply message and hostname of destination
        Socket s = null;
        try{
            int serverPort = 7896;
            s = new Socket(args[1], serverPort);
            DataInputStream in = new DataInputStream( s.getInputStream());
            DataOutputStream out =
                new DataOutputStream( s.getOutputStream());
            out.writeUTF(args[0]);          // UTF is a string encoding see Sn 4.3
            String data = in.readUTF();
            System.out.println("Received: " + data) ;
        }catch (UnknownHostException e){
            System.out.println("Sock:" + e.getMessage());
        }catch (EOFException e){System.out.println("EOF:" + e.getMessage());
        }catch (IOException e){System.out.println("IO:" + e.getMessage());}
    }finally {if(s!=null) try {s.close();}catch (IOException e){System.out.println("close:" + e.getMessage());}}
}
```

TCP SERVER MAKES A CONNECTION FOR EACH CLIENT AND THEN ECHOES THE CLIENT'S REQUEST

```
import java.net.*;
import java.io.*;
public class TCPServer {
    public static void main (String args[]) {
        try{
            int serverPort = 7896;
            ServerSocket listenSocket = new ServerSocket(serverPort);
            while(true) {
                Socket clientSocket = listenSocket.accept();
                Connection c = new Connection(clientSocket);
            }
        } catch(IOException e) {System.out.println("Listen :"+e.getMessage());}
    }
}
```

// this figure continues on the next slide

CONTINUED

```
class Connection extends Thread {
    DataInputStream in;
    DataOutputStream out;
    Socket clientSocket;
    public Connection (Socket aClientSocket) {
        try {
            clientSocket = aClientSocket;
            in = new DataInputStream( clientSocket.getInputStream());
            out =new DataOutputStream( clientSocket.getOutputStream());
            this.start();
        } catch(IOException e) {System.out.println("Connection:"+e.getMessage());}
    }
    public void run(){
        try {                                // an echo server
            String data = in.readUTF();
            out.writeUTF(data);
        } catch(EOFException e) {System.out.println("EOF:"+e.getMessage());}
        } catch(IOException e) {System.out.println("IO:"+e.getMessage());}
        } finally{ try {clientSocket.close();}catch (IOException e){/*close failed*/}}
    }
}
```

EXTERNAL DATA REPRESENTATION AND MARSHALLING

- The individual primitive data items transmitted in messages can be data values of many different types, and not all computers store primitive values such as integers in the same order.
- The representation of floating-point numbers also differs between architectures.
- There are two variants for the ordering of integers: the so-called big-endian order, in which the most significant byte comes first; and little-endian order, in which it comes last.
- To support RMI or RPC, any data type that can be passed as an argument or returned as a result must be able to be flattened and the individual primitive data values represented in an agreed format.
- An agreed standard for the representation of data structures and primitive values is called an external data representation.

EXTERNAL DATA REPRESENTATION AND MARSHALLING

- **Marshalling** is the process of taking a collection of data items and assembling them into a form suitable for transmission in a message.
- **Unmarshalling** is the process of disassembling them on arrival to produce an equivalent collection of data items at the destination.
- **Marshalling** consists of the translation of structured data items and primitive values into an external data representation.
- Similarly, **unmarshalling** consists of the generation of primitive values from their external data representation and the rebuilding of the data structures.

CORBA CDR FOR CONSTRUCTED TYPES

<i>Type</i>	<i>Representation</i>
<i>sequence</i>	length (unsigned long) followed by elements in order
<i>string</i>	length (unsigned long) followed by characters in order (can also have wide characters)
<i>array</i>	array elements in order (no length specified because it is fixed)
<i>struct</i>	in the order of declaration of the components
<i>enumerated</i>	unsigned long (the values are specified by the order declared)
<i>union</i>	type tag followed by the selected member

CORBA CDR MESSAGE

<i>index in sequence of bytes</i>	<i>← 4 bytes →</i>	<i>notes on representation</i>
0–3	5	<i>length of string</i>
4–7	"Smit"	'Smith'
8–11	"h "	
12–15	6	<i>length of string</i>
16–19	"Lond"	'London'
20–23	"on "	
24–27	1984	<i>unsigned long</i>

The flattened form represents a *Person* struct with value: {'Smith', 'London', 1984}

INDICATION OF JAVA SERIALIZED FORM

<i>Serialized values</i>				<i>Explanation</i>
Person	8-byte version number		h0	<i>class name, version number</i>
3	int year	java.lang.String name:	java.lang.String place:	<i>number, type and name of instance variables</i>
1984	5 Smith	6 London	h1	<i>values of instance variables</i>

The true serialized form contains additional type markers; h0 and h1 are handles

XML DEFINITION OF THE PERSON STRUCTURE

```
<person id="123456789">  
  <name>Smith</name>  
  <place>London</place>  
  <year>1984</year>  
  <!-- a comment -->  
</person >
```

ILLUSTRATION OF THE USE OF A NAMESPACE IN THE PERSON STRUCTURE

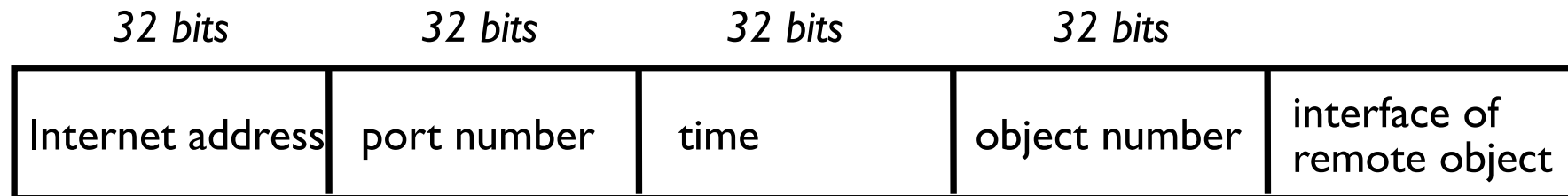
```
<person pers:id="123456789" xmlns:pers = "http://www.cdk5.net/person">  
  <pers:name> Smith </pers:name>  
  <pers:place> London </pers:place >  
  <pers:year> 1984 </pers:year>  
</person>
```

AN XML SCHEMA FOR THE PERSON STRUCTURE

```
<xsd:schema xmlns:xsd = URL of XML schema definitions >
  <xsd:element name= "person" type ="personType" />
  <xsd:complexType name="personType">
    <xsd:sequence>
      <xsd:element name = "name" type="xs:string"/>
      <xsd:element name = "place" type="xs:string"/>
      <xsd:element name = "year" type="xs:positiveInteger"/>
    </xsd:sequence>
    <xsd:attribute name= "id" type = "xs:positiveInteger"/>
  </xsd:complexType>
</xsd:schema>
```

REPRESENTATION OF A REMOTE OBJECT REFERENCE

- A remote object reference is an identifier for a remote object that is valid throughout a distributed system.
- A remote object reference is passed in the invocation message to specify which object is to be invoked.



MULTICAST COMMUNICATION

- A multicast operation is more appropriate – this is an operation that sends a single message from one process to each of the members of a group of processes, usually in such a way that the membership of the group is transparent to the sender.
- There is a range of possibilities in the desired behaviour of a multicast.
- The simplest multicast protocol provides no guarantees about message delivery or ordering.
- Multicast messages provide a useful infrastructure for constructing distributed systems with the **following characteristics**:
- *Fault tolerance based on replicated services*: A replicated service consists of a group of servers. Client requests are multicast to all the members of the group, each of which performs an identical operation. Even when some of the members fail, clients can still be served.

MULTICAST COMMUNICATION

- *Discovering services in spontaneous networking*: defines service discovery in the context of spontaneous networking. Multicast messages can be used by servers and clients to locate available discovery services in order to register their interfaces or to look up the interfaces of other services in the distributed system.
- *Better performance through replicated data*: Data are replicated to increase the performance of a service – in some cases replicas of the data are placed in users' computers. Each time the data changes, the new value is multicast to the processes managing the replicas.
- *Propagation of event notifications*: Multicast to a group may be used to notify processes when something happens. For example, in Facebook, when someone changes their status, all their friends receive notifications.

MULTICAST COMMUNICATION

IP multicast

- IP multicast is built on top of the Internet Protocol (IP).
- Note that IP packets are addressed to computers – ports belong to the TCP and UDP levels. IP multicast allows the sender to transmit a single IP packet to a set of computers that form a multicast group.
- The sender is unaware of the identities of the individual recipients and of the size of the group.
- A multicast group is specified by a Class D Internet address

IP MULTICAST

- ***Multicast routers:*** IP packets can be multicast both on a local network and on the wider Internet. Local multicasts use the multicast capability of the local network, for example, of an Ethernet. Internet multicasts make use of multicast routers, which forward single datagrams to routers on other networks, where they are again multicast to local members. To limit the distance of propagation of a multicast datagram, the sender can specify the number of routers it is allowed to pass – called the time to live, or TTL for short.
- ***Multicast address allocation:*** As discussed in Chapter 3, Class D addresses (that is, addresses in the range 224.0.0.0 to 239.255.255.255) are reserved for multicast traffic and managed globally by the Internet Assigned Numbers Authority (IANA).

MULTICAST COMMUNICATION

Failure model for multicast datagrams

- Datagrams multicast over IP multicast have the same failure characteristics as UDP datagrams – that is, they suffer from omission failures.
- The effect on a multicast is that messages are not guaranteed to be delivered to any particular group member in the face of even a single omission failure.
- That is, some but not all of the members of the group may receive it.
- This can be called unreliable multicast, because it does not guarantee that a message will be delivered to any member of a group.

MULTICAST PEER JOINS A GROUP AND SENDS AND RECEIVES DATAGRAMS

```
import java.net.*;
import java.io.*;
public class MulticastPeer{
    public static void main(String args[]){
        // args give message contents & destination multicast group (e.g. "228.5.6.7")
        MulticastSocket s =null;
        try {
            InetAddress group = InetAddress.getByName(args[1]);
            s = new MulticastSocket(6789);
            s.joinGroup(group);
            byte [] m = args[0].getBytes();
            DatagramPacket messageOut =
                new DatagramPacket(m, m.length, group, 6789);
            s.send(messageOut);
```

// this figure continued on the next slide

CONTINUED

```
// get messages from others in group
    byte[] buffer = new byte[1000];
    for(int i=0; i< 3; i++) {
        DatagramPacket messageIn =
            new DatagramPacket(buffer, buffer.length);
        s.receive(messageIn);
        System.out.println("Received:" + new String(messageIn.getData()));
    }
    s.leaveGroup(group);
    }catch (SocketException e){System.out.println("Socket: " + e.getMessage());
    }catch (IOException e){System.out.println("IO: " + e.getMessage());}
    }finally {if(s != null) s.close();}
    }
}
```

MULTICAST COMMUNICATION

Java API to IP multicast

- The Java API provides a datagram interface to IP multicast through the class `MulticastSocket`, which is a subclass of `DatagramSocket` with the additional capability of being able to join multicast groups.
- The class `MulticastSocket` provides two alternative constructors, allowing sockets to be created to use either a specified local port (6789, in Figure 4.14) or any free local port.

Reliability and ordering of multicast

- Ordering is another issue.
- IP packets sent over an internetwork do not necessarily arrive in the order in which they were sent, with the possible effect that some group members receive datagrams from a single sender in a different order from other group members.
- In addition, messages sent by two different processes will not necessarily arrive in the same order at all the members of the group.

NETWORK VIRTUALIZATION: OVERLAY NETWORKS

- Network virtualization [Petersen et al. 2005] is concerned with the construction of many different virtual networks over an existing network such as the Internet.
- Each virtual network can be designed to support a particular distributed application.
- An application-specific virtual network can be built above an existing network and optimized for that particular application, without changing the characteristics of the underlying network.

NETWORK VIRTUALIZATION: OVERLAY NETWORKS

Overlay networks:

- An overlay network is a virtual network consisting of nodes and virtual links, which sits on top of an underlying network (such as an IP network) and offers something that is not otherwise provided:
 - a service that is tailored towards the needs of a class of application or a particular higher-level service – for example, multimedia content distribution;
 - more efficient operation in a given networked environment – for example routing in an ad hoc network;
 - an additional feature – for example, multicast or secure communication.

NETWORK VIRTUALIZATION: OVERLAY NETWORKS

Overlay networks have the following advantages:

- They enable new network services to be defined without requiring changes to the underlying network, a crucial point given the level of standardization in this area and the difficulties of amending underlying router functionality.
- They encourage experimentation with network services and the customization of services to particular classes of application.
- Multiple overlays can be defined and can coexist, with the end result being a more open and extensible network architecture.

NETWORK VIRTUALIZATION: OVERLAY NETWORKS

Overlay networks have the following advantages:

- They enable new network services to be defined without requiring changes to the underlying network, a crucial point given the level of standardization in this area and the difficulties of amending underlying router functionality.
- They encourage experimentation with network services and the customization of services to particular classes of application.
- Multiple overlays can be defined and can coexist, with the end result being a more open and extensible network architecture.

NETWORK VIRTUALIZATION: OVERLAY NETWORKS

Overlay networks have the following advantages:

- They enable new network services to be defined without requiring changes to the underlying network, a crucial point given the level of standardization in this area and the difficulties of amending underlying router functionality.
- They encourage experimentation with network services and the customization of services to particular classes of application.
- Multiple overlays can be defined and can coexist, with the end result being a more open and extensible network architecture.

NETWORK VIRTUALIZATION: OVERLAY NETWORKS

The disadvantages are:

- overlays introduce an extra level of indirection (and hence may incur a performance penalty)
- they add to the complexity of network services when compared, for example, to the relatively simple architecture of TCP/IP networks

TYPES OF OVERLAY

<i>Motivation</i>	<i>Type</i>	<i>Description</i>
<i>Tailored for application needs</i>	Distributed hash tables	One of the most prominent classes of overlay network, offering a service that manages a mapping from keys to values across a potentially large number of nodes in a completely decentralized manner (similar to a standard hash table but in a networked environment).
	Peer-to-peer file sharing	Overlay structures that focus on constructing tailored addressing and routing mechanisms to support the cooperative discovery and use (for example, download) of files.
	Content distribution networks	Overlays that subsume a range of replication, caching and placement strategies to provide improved performance in terms of content delivery to web users; used for web acceleration and to offer the required real-time performance for video streaming [www.kontiki.com].

(CONTINUED) TYPES OF OVERLAY

*Tailored for
network style*

Wireless ad hoc
networks

Network overlays that provide customized routing protocols for wireless ad hoc networks, including proactive schemes that effectively construct a routing topology on top of the underlying nodes and reactive schemes that establish routes on demand typically supported by flooding.

Disruption-tolerant
networks

Overlays designed to operate in hostile environments that suffer significant node or link failure and potentially high delays.

*Offering additional
features*

Multicast

One of the earliest uses of overlay networks in the Internet, providing access to multicast services where multicast routers are not available; builds on the work by Van Jacobsen, Deering and Casner with their implementation of the MBone (or Multicast Backbone) [[mbone](http://mbone.org)].

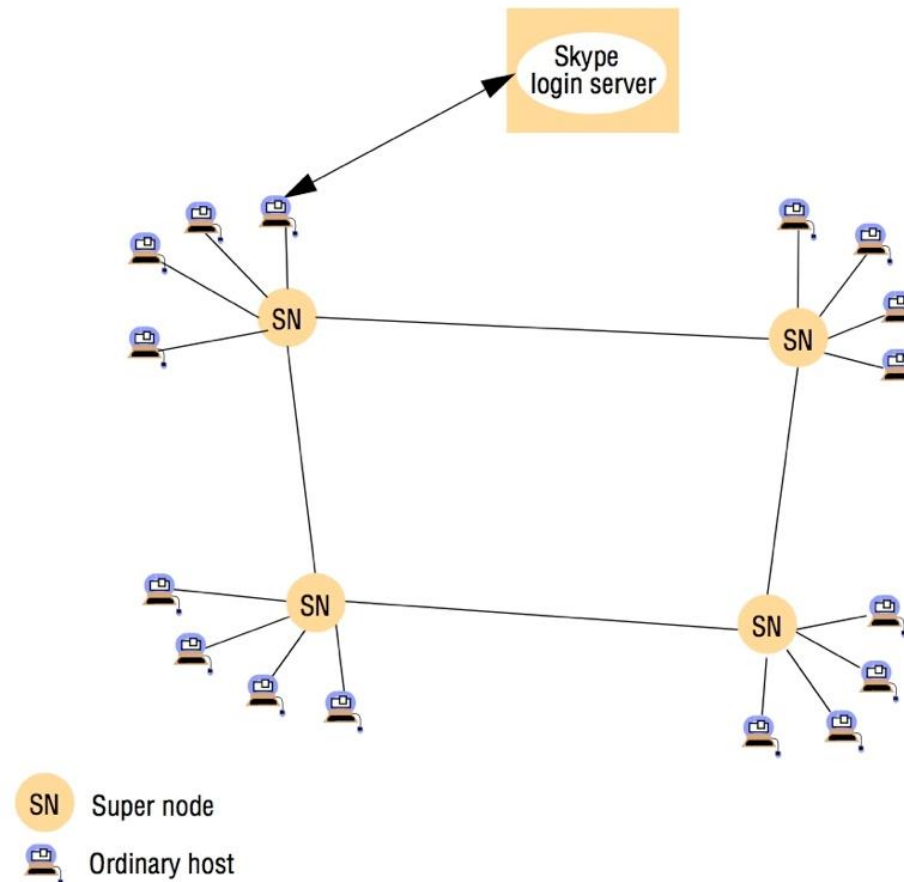
Resilience

Overlay networks that seek an order of magnitude improvement in robustness and availability of Internet paths [nms.csail.mit.edu].

Security

Overlay networks that offer enhanced security over the underlying IP network, including virtual private networks, for example, as discussed in Section 3.4.8.

SKYPE OVERLAY ARCHITECTURE



REFERENCES

- Instructor's Guide for Coulouris, Dollimore, Kindberg and Blair, *Distributed Systems: Concepts and Design* Edn. 5 © Pearson Education 2012
- Text extensions to slides © David E. Bakken, 2012-2020
- **Distributed Systems: Concepts and Design**, Coulouris, Dollimore, Kindberg and Blair, Edn. 5 © Pearson Education 2012